



FOR SPRING DEVELOPERS

Your thread just changed.

Spring Boot to Python (FastAPI): where your code runs, on one payments service built twice

Your instinct is right about the shape of the code and wrong about where it runs.

One small payments service, built twice, layer for layer: a Spring Boot controller, service, pricing component, provider client and store, against a FastAPI router, service, pricing function, httpx provider client and SQLAlchemy store. Same SQL, same rules, the same slow payment provider.

Spring MVC gives you strong defaults for where your code runs, how long your objects live and where your transactions start. FastAPI makes each of those boundaries explicit, and one keyword, `def` or `async def`, decides which thread your request runs on.

At the time of writing, October 2026: Python 3.14.8, FastAPI 0.142.2 (still before 1.0), Pydantic 2.13.5, Uvicorn 0.54.0, SQLAlchemy 2.1.4 and psycopg 3.3.6, against Spring Boot 4.1.1 on Java 25. Newer versions may differ.

This service exercised

- Type hints against Java types, and validation at the API boundary
- `def` against `async def`, and AnyIO's thread limit
- Dependency lifetime, lifespan objects and worker processes
- SQLAlchemy transactions inside a FastAPI dependency, and BackgroundTasks

Not covered

- Alembic migrations, settings and testing patterns
- SQLAlchemy's async engine
- Footprint and startup comparisons
- Gunicorn, containers and Kubernetes tuning

Every mistake is an overlay in the repository, laid over a scratch copy of an app, and `scripts/verify.sh` measures all of them on your machine.

01 The boundary ladder

Request: where the handler runs	Spring: a Tomcat request thread (200 by default). FastAPI: async def on the event loop, def on an AnyIO thread (40 at once by default)
Service: how long objects live	Spring: singleton bean by default. FastAPI: a dependency per request, a lifespan object per process
Database: where the transaction ends	Spring: @Transactional, when the method returns. FastAPI: SQLAlchemy session.begin(), ended by the dependency scope
Instance: work after the response	Spring: in-memory @Async dies with the JVM. FastAPI: BackgroundTasks dies with the worker
Scale out: copies of memory	Spring: one per JVM, so one per replica. FastAPI: one per Uvicorn worker, --workers 4 is four

Measured, not asserted: AnyIO's default limiter is 40 tokens and Tomcat's default is 200 threads (evidence/defaults.log); 40 blocking def calls took 2.12 s and the 41st took 4.1 s; a dependency was built once per request, a lifespan object once per process; four workers, four copies of memory.

02 One word. Ten times slower.

Both provider clients block for the two seconds the provider takes. In Spring that blocks one request thread. Inside async def it blocks the event loop, so every other request on that worker waits, the health check included.

Spring: ProviderClient.java (excerpt)

```
public void charge(String orderRef, long amount) {
    http.post().uri("/charges")
        .body(Map.of("order_ref", orderRef,
                     "amount", amount))
        .retrieve().toBodilessEntity();
}
```

FastAPI: overlays/async-blocking, payments/api.py (excerpt)

```
@router.post("/payments", status_code=201)
async def create(
    new: NewPayment,
    tasks: BackgroundTasks,
    service: PaymentService =
    Depends(payment_service),
    receipt_sender: Receipts = Depends(receipts),
) -> Payment:
    payment = service.create(new)
```

Ten payments at once (evidence/hook-*.log)

```
FastAPI, async def: 10 payments in 20.32 s, /health answered in 18.08 s, status codes {201: 10}
FastAPI, def:       10 payments in 2.1 s, /health answered in 0.0 s, status codes {201: 10}
Spring:             10 payments in 2.26 s, /health answered in 0.0 s, status codes {201: 10}
```

The labels on the left are added; each result is copied from its own transcript.

A blocking client inside async def: 10 payments 20.32 s, /health 18.08 s. The same endpoint as def: 2.1 s. Spring RestClient against the same provider: 2.26 s.

Measured on a real run, 2026-10-07: evidence/ in the repository, reproduced by scripts/verify.sh.

03 These look like Java types. They are not.

Python annotations are read by tools, not enforced when the code runs. Run mypy in the build. At the HTTP boundary, Pydantic does use them, and the error is a 422, not Spring's 400: a change to your API contract if you are porting a service.

Spring: Pricing.java (excerpt)

```
public long total(long unitPrice, int quantity)
{
    return unitPrice * quantity;
}
```

FastAPI: payments/pricing.py

```
def total(unit_price: int, quantity: int) -> int:
    return unit_price * quantity
```

total("5", 3) on each side (evidence/types.log, excerpt)

```
experiments/TypesChecked.java:10: error: incompatible types: String cannot be converted to long
```

```
$ PYTHONPATH=fastapi-payments python experiments/types_checked.py
'555'
```

```
experiments/types_checked.py:3: error: Argument 1 to "total" has incompatible type "str";
expected "int" [arg-type]
```

Spring: NewPayment.java (excerpt)

```
public record NewPayment(
    @NotBlank String orderRef,
    @Positive long unitPrice,
    @Positive int quantity,
    @Size(min = 3, max = 3) String currency) {
}
```

FastAPI: payments/schemas.py (excerpt)

```
class NewPayment(BaseModel):
    order_ref: str = Field(min_length=1)
    unit_price: int = Field(gt=0)
    quantity: int = Field(gt=0)
    currency: str = Field(min_length=3,
                          max_length=3)
```

The string "5" for unit_price (evidence/validation-coercion.log)

```
{"id":22,"order_ref":"order-5","total":15,"currency":"GBP","status":"CAPTURED"} HTTP 201
```

Pydantic's lax mode turned "5" into 5 at the boundary: a total of 15, not '555'.

A negative unit price (evidence/validation.log)

```
{"detail":[{"type":"greater_than","loc":["body","unit_price"],"msg":"Input should be greater
than 0","input":-5,"ctx":{"gt":0}}]}
HTTP 422
```

```
{"timestamp":"2026-10-07T19:26:45.903Z","status":400,"error":"Bad Request","path":"/payments"}
HTTP 400
```

javac refuses `total("5", 3)`; Python returns '555'; mypy rejects it. Through the API, "5" becomes 5 and the total is 15. An invalid body is 422 in FastAPI and 400 in Spring.

Measured on a real run, 2026-10-07: `evidence/` in the repository, reproduced by `scripts/verify.sh`.

04 def or async def decides the thread

Two clean fixes: a plain `def`, which FastAPI runs on a worker thread instead of the loop, or an `async` client with an `await`, which frees the loop while the provider thinks. What you must not do is put blocking work directly on the event loop. This is an event loop problem, not a GIL one: Python 3.14's optional free-threaded build does not rescue it.

FastAPI: `experiments/fastapi_threads.py`
(excerpt), fix one

```
@app.post("/payments/blocking")
def blocking() -> dict[str, str]:
    with httpx.Client(timeout=30) as client:
        return client.post(f"
{PROVIDER}/charges").json()
```

FastAPI: `experiments/fastapi_threads.py`
(excerpt), fix two

```
@app.post("/payments/awaited")
async def awaited() -> dict[str, str]:
    async with httpx.AsyncClient(timeout=30) as
client:
        sent = await client.post(f"
{PROVIDER}/charges")
        return sent.json()
```

Ten calls each, no database in the way (`evidence/threads-fastapi.log`, excerpt)

```
$ load http://127.0.0.1:8096 /payments/blocking-in-async 10
10 payments in 20.17 s, /health answered in 19.95 s, status codes {200: 10}
$ load http://127.0.0.1:8096 /payments/awaited 10
10 payments in 2.1 s, /health answered in 0.0 s, status codes {200: 10}
$ load http://127.0.0.1:8096 /payments/blocking 10
10 payments in 2.06 s, /health answered in 0.0 s, status codes {200: 10}
```

Blocking in `async def`: 20.17 s. Awaited `AsyncClient`: 2.1 s. Plain `def`: 2.06 s.

Measured on a real run, 2026-10-07: `evidence/` in the repository, reproduced by `scripts/verify.sh`.

05 Forty took two seconds. Forty one took four.

FastAPI hands plain def endpoints to Starlette, which runs them on AnyIO's thread pool, and AnyIO's default limiter allows 40 at once. The limit is AnyIO's, not FastAPI's. It is one line to raise, but the same limiter is shared with your synchronous dependencies, and in the full app the database connection pool is the next queue. Raising one limit moves the queue. It does not remove it.

Spring: experiments/spring-threads (excerpt)

```
@PostMapping("/payments/blocking")
Map<?, ?> blocking() {
    return provider.post().uri("/charges")
        .retrieve().body(Map.class);
}
```

FastAPI: overlays/limiter-100, payments/main.py (excerpt)

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    limiter =
to_thread.current_default_thread_limiter()
limiter.total_tokens = 100
```

The two defaults, read from the installed versions (evidence/defaults.log)

```
AnyIO default thread limiter: 40 tokens
Spring Boot 4.1.1, server.tomcat.threads.max default: 200
```

Seconds for N blocking calls at once (evidence/threads-*.log)

N	40	41	100
FastAPI def	2.12	4.1	6.25
FastAPI, limit 100	2.14	2.14	2.29
Spring MVC	2.1	2.09	2.24

Arranged as a table; every figure is copied from the transcripts. /health answered within 0.01 s in every run.

Default limiter: 40 calls 2.12 s, 41 calls 4.1 s, 100 calls 6.25 s. Raised to 100 tokens: 2.14, 2.14 and 2.29 s. Defaults recorded in the run: "AnyIO default thread limiter: 40 tokens", "server.tomcat.threads.max default: 200".

Measured on a real run, 2026-10-07: evidence/ in the repository, reproduced by scripts/verify.sh.

06 How long does this object live?

In Spring, singleton is the default bean scope. In FastAPI, a Depends function is called on every request, its value cached within that request. Build expensive clients once, in the lifespan, and keep them in app state. But once means once per process.

Spring: ProviderClient.java (excerpt)

```
@Component
public class ProviderClient {

    private final RestClient http;

    public ProviderClient(
        @Value("${provider.url}") String url) {
        this.http = RestClient.create(url);
    }
}
```

FastAPI: payments/main.py (excerpt)

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.sessions = db.connect()
    app.state.provider =
    ProviderClient(PROVIDER_URL)
    yield
    app.state.provider.close()
```

Three requests to experiments/lifetime.py (evidence/lifetime.log, excerpt)

```
{"same_object_twice_in_one_request":true,"built":{"per_request":1,"lifespan":1},...}
{"same_object_twice_in_one_request":true,"built":{"per_request":2,"lifespan":1},...}
{"same_object_twice_in_one_request":true,"built":{"per_request":3,"lifespan":1},...}
```

Shortened with "..."; the pid and object id are in the transcript.

Five payments, counted by the provider (evidence/lifetime-*.log)

```
provider client built per request: {"charges":5,"distinct_connections":5}
provider client built in the lifespan: {"charges":5,"distinct_connections":1}
spring, one ProviderClient bean: {"charges":5,"distinct_connections":1}

worker processes seen: 4  lifespan objects seen: 4
```

Per request: 1, 2, 3 gateways built, the same object twice within one request, one lifespan object. With --workers 4: four processes, four lifespan objects.

Measured on a real run, 2026-10-07: evidence/ in the repository, reproduced by scripts/verify.sh.

07 201 Created. Zero rows.

Three steps. 1: a session from a yield dependency with no transaction. Closing an uncommitted session rolls back silently: 201, and the row never exists. 2: add SQLAlchemy's boundary, `session.begin()`. Now it commits, but after the response, because a yield dependency defaults to the request scope. 3: `Depends(get_session, scope="function")` commits before the response, which matches Spring's `@Transactional`.

The commit is SQLAlchemy's. When it happens is FastAPI's. Spring does not wrap your code in a transaction either, unless you declare one.

Step 1: overlays/no-transaction,
payments/deps.py (excerpt)

```
def get_session(request: Request) ->
    Iterator[Session]:
    with request.app.state.sessions() as session:
        yield session
```

Step 2: overlays/commit-after-response,
payments/deps.py (excerpt)

```
def get_session(request: Request) ->
    Iterator[Session]:
    with request.app.state.sessions() as session:
        with session.begin():
            yield session

    session: Session = Depends(get_session),
```

Spring: PaymentService.java (excerpt)

```
@Transactional
public Payment create(NewPayment in) {
    long amount = pricing.total(
        in.unitPrice(), in.quantity());
    Optional<Long> found =
    store.find(in.orderRef());
```

Step 3: payments/deps.py (excerpt)

```
def payment_service(
    request: Request,
    session: Session = Depends(
        get_session, scope="function"
    ),
) -> PaymentService:
```

Rows in Postgres after HTTP 201 (evidence/transaction-*.log)

step 1, no begin:	right after the 201: payments: 0	five seconds later: payments: 0
step 2, request scope:	right after the 201: payments: 0	five seconds later: payments: 1
step 3, function scope:	right after the 201: payments: 1	five seconds later: payments: 1
spring @Transactional:	right after the 201: payments: 1	five seconds later: payments: 1

One payment per case, each answered HTTP 201. The case labels are added and the receipt counts left out; the full lines are in the transcripts.

Why step 2 commits late (evidence/fastapi-scope.log, excerpt)

```
* `request`: start the dependency before the *path operation
function* that handles the request (similar to when using
`function`), but end after the response is sent back to
the client. So, the dependency function will be executed
around the request and response cycle.
```

No begin: 201 and 0 rows, still 0 five seconds later. Request scope: 0 rows right after the 201, 1 five seconds later. Function scope and Spring: 1 row right after the 201.

Measured on a real run, 2026-10-07: `evidence/` in the repository, reproduced by `scripts/verify.sh`.

The "function" scope ends the dependency before the response is sent; the "request" scope ends it after.

FastAPI 0.142.2, the `Depends(scope=...)` docstring, printed by `experiments/scope_doc.py` into `evidence/fastapi-scope.log`.

08 Work after the response is not a queue

BackgroundTasks runs the receipt after the response has gone, in the worker's memory. Kill the worker half a second after the 201 and the receipt never arrives. Spring's in-memory @Async loses it the same way when the JVM dies. If the work matters, put it in an outbox table or a message broker.

Spring: `ReceiptService.java` (excerpt)

```
@Async
public void send(long paymentId)
    throws InterruptedException {
    Thread.sleep(2000);
}
```

FastAPI: `payments/api.py` (excerpt)

```
payment = service.create(new)
tasks.add_task(receipt_sender.send, payment.id)
return payment
```

`evidence/background.log` and `background-spring.log` (excerpt)

```
not killed, receipts: 1
worker killed half a second later, receipts: 1
spring @Async, JVM killed, receipts: 0
```

The FastAPI count stays at 1: the first payment's receipt arrived, the second never did.

Each killed process answered HTTP 201 and its receipt was never written.

Measured on a real run, 2026-10-07: `evidence/` in the repository, reproduced by `scripts/verify.sh`.

09 Workers are processes

uvicorn --workers 4 is four processes, each with its own copy of memory. With an in-memory idempotency dictionary, the same order sent eight times was charged seven times. Several Spring replicas have the same problem; what differs is that one flag creates the boundary on one host.

● ● ● Spring: overlays/in-memory-idempotency (excerpt)

```
private final Map<String, Long> seen =  
    new ConcurrentHashMap<>();
```

● ● ● FastAPI: overlays/in-memory-idempotency (excerpt)

```
seen: dict[str, int] = {}
```

● ● ● order-42, eight times, one after another (evidence/workers-*.log, excerpt)

```
FastAPI, --workers 4: HTTP 201 twice, HTTP 500 six times  
provider: {"charges":7,"distinct_connections":4}  
rows in payments: 1
```

```
Spring, one JVM: HTTP 201 eight times  
provider: {"charges":1,"distinct_connections":1}
```

The status lines are summarised; each HTTP line is in the transcript. The unique column refused the duplicate rows, after the provider had charged them. The second 201 landed on the worker that had already inserted the row, so it answered without charging.

Four workers: 7 charges over 4 connections, 1 row, 6 server errors. One Spring JVM: 1 charge.

Measured on a real run, 2026-10-07: evidence/ in the repository, reproduced by scripts/verify.sh.



Starting a FastAPI service, as a Spring developer

1. Run mypy in the build: the runtime does not enforce your annotations
2. Decide your error contract: FastAPI validation answers 422, Spring answers 400
3. Never call a blocking client inside async def: use def, or an async client with await
4. Know the thread budget: AnyIO allows 40 def calls at once by default, shared with sync dependencies
5. Build expensive clients once, in the lifespan; a Depends function runs on every request
6. Wrap the session in session.begin() and use Depends(..., scope="function") so the commit lands before the response
7. Assert the database in your tests, not just the status code: a 201 can have no row behind it
8. Put work that matters in an outbox or a broker, not BackgroundTasks or in-memory @Async
9. Treat every Uvicorn worker as a separate instance: no in-memory state you rely on for correctness

Everything else

-  **Site**
<https://code-with-sam-dev.github.io>
-  **YouTube**
<https://www.youtube.com/@CodewithSam-Dev>
-  **LinkedIn**
<https://www.linkedin.com/company/code-with-sam-dev>
-  **x**
<https://x.com/CodeWithSamDev>
-  **TikTok**
<https://www.tiktok.com/@codewithsamdev>
-  **Instagram**
<https://www.instagram.com/codewithsamdev>
-  **Facebook**
<https://www.facebook.com/codewithsamdev>
-  **GitHub**
<https://github.com/code-with-sam-dev>
-  **Buy me a coffee**
<https://buymeacoffee.com/codewithsam>
-  **Buy me sadza**
<https://buymesadza.com/codewithsam>
The repository: both services, the overlays, and scripts/verify.sh
<https://github.com/code-with-sam-dev/spring-to-python>
FastAPI: concurrency and async / await
<https://fastapi.tiangolo.com/async/>
FastAPI: dependencies with yield
<https://fastapi.tiangolo.com/tutorial/dependencies/dependencies-with-yield/>
AnyIO: working with threads
<https://anyio.readthedocs.io/en/stable/threads.html>
SQLAlchemy: transactions and connection management
https://docs.sqlalchemy.org/en/latest/orm/session_transaction.html
Uvicorn: deployment
<https://www.uvicorn.org/deployment/>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which stack you want Spring Boot compared with next.

Java is a trademark of Oracle. Spring and Spring Boot are trademarks of Broadcom. Python is a trademark of the Python Software Foundation. PostgreSQL is a trademark of the PostgreSQL Community Association. FastAPI, Starlette, AnyIO, Pydantic, Uvicorn, SQLAlchemy, httpx and mypy are the property of their respective owners. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-10-07 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.